PTC Interview Preparation Guide

Senior DevOps Engineer

PTC Inc. — Interview Q&A; Reference
Round 1: Conceptual | Round 2: Scenario-Based

■ Round 1 — Conceptual Questions

Q1. Explain your project architecture and CI/CD pipeline.

Structure your answer in 3 layers: Infrastructure → Application → Automation.

Example (ApnaKart-style):

- Infrastructure: AWS EKS cluster provisioned via Terraform with S3 remote state. Multi-environment: dev / test / uat / prod.
- Application: 10 Spring Boot microservices + React frontend, containerised with Docker, pushed to ECR.
- CI/CD: Jenkins pipelines triggered on branch push. Branch → environment mapping (main→prod, develop→uat). SonarQube for code quality, Trivy for image scan, Helm/Kustomize for K8s deployment. Datadog APM for observability.

■ *Interview tip: Speak to a diagram in your head. Mention quantified outcomes: '60% faster deployments, 99.9% availability.'*

Q2. What is Docker? Difference between image and container?

Docker is a containerisation platform that packages an application with its dependencies into a portable, isolated unit.

Image vs Container:

- Image: Read-only template built from a Dockerfile. Stored in a registry (ECR, Docker Hub). Blueprint.
- Container: A running instance of an image. Has a writable layer on top. Ephemeral by default.
- Analogy: Image = class definition. Container = object instance.

■ *Interview tip: Mention layers — images are built in cacheable layers. Changing a layer invalidates all layers below it.*

Q3. What is Kubernetes? Explain its architecture.

Kubernetes (K8s) is an open-source container orchestration system that automates deployment, scaling, and management of containerised applications.

Control Plane (master):

- API Server: Single entry point. All kubectl commands hit this.
- etcd: Distributed key-value store. Stores entire cluster state.
- Scheduler: Assigns pods to nodes based on resource availability and constraints.
- Controller Manager: Runs controllers (ReplicaSet, Deployment, Node) that reconcile desired vs actual state.

Worker Node:

- kubelet: Agent on each node. Ensures containers in pods are running.
- kube-proxy: Handles network rules and service routing on the node.
- Container Runtime: Docker / containerd — actually runs containers.

■ *Interview tip: Draw this mentally as two boxes: Control Plane and Worker Node, with arrows showing kubelet←→API Server communication.*

Q4. What is a Pod, Service, and Deployment?

- Pod: Smallest deployable unit. One or more containers sharing network/storage. Gets a unique IP per pod.
- Service: Stable network endpoint that load-balances traffic to a set of pods using label selectors. Types: ClusterIP, NodePort, LoadBalancer.
- Deployment: Manages ReplicaSets. Declares desired state (replicas, image). Handles rolling updates and rollbacks.

■ *Interview tip: A Service decouples consumers from pod churn — pods die and restart with new IPs, but the Service IP stays constant.*

Q5. Difference between Rolling, Blue-Green, and Canary deployments.

- Rolling: Replace old pods with new ones incrementally. Zero downtime but both versions coexist briefly. Default K8s strategy.
- Blue-Green: Two identical environments (blue=live, green=new). Switch traffic all at once via load balancer. Instant rollback by flipping back. High resource cost.
- Canary: Route a small % of traffic (e.g., 5%) to new version. Gradually increase if metrics are healthy. Best for risk-controlled releases.

■ *Interview tip: Canary is safest for production. Blue-Green is fastest to rollback. Rolling is cheapest on resources.*

Q6. What is Terraform? What is state file?

Terraform is an Infrastructure-as-Code tool by HashiCorp that provisions cloud resources using declarative HCL configuration.

State file (terraform.tfstate):

- Maps real-world resources to your config. Terraform uses it to know what exists, what changed, what to destroy.
- Stored locally by default — but must be remote (S3 + DynamoDB) for team use.
- Never edit manually. If corrupted, infrastructure drift occurs.
- Sensitive — contains resource IDs, IPs, sometimes secrets. Encrypt at rest.

■ *Interview tip: Always mention remote state with locking (DynamoDB) — it prevents concurrent apply conflicts.*

Q7. Difference between public and private subnets in AWS.

- Public Subnet: Has a route to an Internet Gateway (IGW). Resources (e.g., ALB, NAT Gateway, bastion host) are reachable from the internet.
- Private Subnet: No direct route to IGW. Resources (e.g., EKS nodes, RDS, Lambda) are isolated. Outbound internet access via NAT Gateway in public subnet.
- Best practice: Place application workloads in private subnets. Only expose what must be public (load balancers).

■ *Interview tip: Mention: A subnet is 'public' only because of its route table entry pointing to an IGW — not because of anything inherent.*

Q8. What is IAM? Difference between role and policy?

IAM (Identity and Access Management) controls who can do what in AWS. It manages authentication (who you are) and authorisation (what you can do).

Policy vs Role:

- Policy: JSON document defining permissions (Allow/Deny on Actions and Resources). Attached to identities.
- Role: An IAM identity with no permanent credentials. Assumed by AWS services (EC2, Lambda, EKS pods via IRSA), users, or accounts. Provides temporary credentials.
- Key difference: A user has long-term credentials. A role is assumed temporarily and is preferred for services.

■ *Interview tip: Mention IRSA (IAM Roles for Service Accounts) for pod-level AWS permissions in EKS — it's the secure way to avoid storing AWS keys in pods.*

Q9. What is a Load Balancer and how does it work?

A Load Balancer distributes incoming traffic across multiple targets (EC2, pods, IPs) to ensure high availability and prevent single points of failure.

AWS Load Balancer types:

- ALB (Application LB): Layer 7. Routes based on HTTP path/host. Best for microservices and container workloads.
- NLB (Network LB): Layer 4. Ultra-low latency. TCP/UDP. Used for high-performance workloads.
- CLB (Classic): Legacy. Avoid.
- Health checks: LB continuously probes targets. Unhealthy targets are removed from rotation automatically.

■ *Interview tip: In EKS, an Ingress controller (AWS Load Balancer Controller) creates ALBs automatically from Ingress resources.*

Q10. What is Git branching strategy?

A branching strategy defines how teams create, merge, and manage branches to coordinate development and releases.

Common strategies:

- GitFlow: main, develop, feature/*, release/*, hotfix/* branches. Good for versioned releases.
- Trunk-Based Development: Short-lived feature branches merged to main frequently. Preferred with CI/CD.
- Branch-per-environment: feature→dev, develop→test/uat, main→prod. Maps deployment targets directly.

In ApnaKart:

- Branch-per-environment strategy. Each push to a branch triggers the Jenkins pipeline for that environment.

■ *Interview tip: Be ready to defend your choice. Why not trunk-based? Answer: team coordination across environments, explicit promotion gates before production.*

Q11. How do you monitor applications in production?

Three pillars of observability:

- Metrics: CPU, memory, request rate, error rate, latency. Tools: Datadog, Prometheus + Grafana, CloudWatch.
- Logs: Structured application logs aggregated centrally. Tools: Datadog Log Management, ELK Stack, CloudWatch Logs.
- Traces: Distributed tracing across microservices. Tools: Datadog APM, Jaeger, AWS X-Ray.

In ApnaKart:

- Datadog DaemonSet deployed on EKS nodes. APM traces across 10 microservices. Alerts on p99 latency and error rate SLOs.

■ *Interview tip: Mention SLOs and alerting strategy — not just tooling. 'We alert when error rate crosses 1% over 5 min' is more senior than 'we use Datadog.'*

Q12. What is DNS and how does it work?

DNS (Domain Name System) translates human-readable domain names (e.g., api.myapp.com) into IP addresses that computers use to route traffic.

Resolution flow:

- 1. Browser checks local cache → OS cache → Recursive resolver (ISP/Google 8.8.8.8).
- 2. Recursive resolver queries Root nameserver → TLD nameserver (.com) → Authoritative nameserver.
- 3. Authoritative nameserver returns the IP. Response is cached based on TTL.

AWS context:

- Route 53 is AWS's authoritative DNS. Supports routing policies: Simple, Weighted, Failover, Latency-based, Geolocation.

■ *Interview tip: Mention TTL strategy: low TTL (60s) during migrations for quick failover; high TTL (300s+) in stable production to reduce resolver load.*

■ Round 2 — Scenario-Based Questions

Q1. How do you scale an application in Kubernetes?

Horizontal Scaling (HPA):

- HPA scales pod count based on CPU/memory or custom metrics (Datadog, Prometheus). Define minReplicas, maxReplicas, targetCPUUtilizationPercentage.
- kubectl autoscale deployment my-app --cpu-percent=70 --min=2 --max=10

Vertical Scaling (VPA):

- Adjusts CPU/memory requests/limits for pods. Requires pod restart — less preferred for stateless apps.

Cluster Autoscaler:

- Adds/removes EC2 nodes when pods are unschedulable. Works with AWS Auto Scaling Groups.

■ *Interview tip: Mention Karpenter as the modern alternative to Cluster Autoscaler — faster node provisioning with flexible instance selection.*

Q2. How do you secure container images in Kubernetes?

- Image scanning: Trivy or ECR native scanning — scan in CI pipeline, fail build on CRITICAL CVEs.
- Use minimal base images: distroless or alpine. Reduces attack surface.
- Run as non-root: Set securityContext.runAsNonRoot: true in pod spec.
- Read-only filesystem: readOnlyRootFilesystem: true where possible.
- Image signing: Cosign + Sigstore or AWS Signer. Verify at admission with OPA Gatekeeper or Kyverno.
- Private registry: Pull only from ECR. Disable public image pulls via admission webhook.

■ *Interview tip: In ApnaKart, Trivy scans ECR images in the Jenkins pipeline. Zero critical CVEs in production is a concrete outcome worth stating.*

Q3. What is the role of Amazon CloudFront in an architecture?

CloudFront is AWS's CDN (Content Delivery Network). It caches content at 450+ edge locations globally, reducing latency and origin load.

Use cases:

- Static asset delivery: JS, CSS, images from S3 — served from nearest edge.
- API acceleration: Cache GET responses at edge. Reduce EKS/ALB load.
- HTTPS termination with ACM certificates.
- WAF integration: Block malicious requests at the edge before they hit your origin.
- Geo-restriction: Block traffic from specific countries.

■ *Interview tip: In ApnaKart: CloudFront sits in front of two S3 buckets (frontend assets + static files) with ACM cert. Mention cache invalidation strategy on each deploy.*

Q4. How do you design disaster recovery for Amazon RDS?

Key concepts — RPO (data loss tolerance) and RTO (downtime tolerance):

- Multi-AZ Deployment: Synchronous standby replica in another AZ. Automatic failover in 60–120s. RTO: ~2min, RPO: ~0. Best for HA within a region.
- Read Replicas: Asynchronous. Can be cross-region. Promoted manually in disaster. RTO: minutes, RPO: seconds of lag.
- Automated Backups: Daily snapshots + transaction logs. Restore to point-in-time. Retained up to 35 days.
- Cross-region snapshot copy: For full regional disaster. Highest RTO but protects against region-wide failure.

Strategy by criticality:

- Critical (RPO<1min): Multi-AZ + cross-region Read Replica.
- Standard (RPO<1hr): Multi-AZ + automated backups.

■ *Interview tip: Always frame DR around RPO and RTO numbers. A Senior engineer speaks in SLAs, not just tool names.*

Q5. A developer cannot access an S3 bucket. What will you check?

Systematic checklist — layer by layer:

- 1. IAM: Does the user/role have s3:GetObject (or relevant action) on the bucket ARN? Check attached policies and group memberships.
- 2. Bucket Policy: Does the bucket policy explicitly Deny the user? A Deny overrides any Allow.
- 3. S3 Block Public Access: If they're trying public access, are block settings overriding?
- 4. Bucket ACL: Legacy — is ACL restricting access?
- 5. VPC Endpoint: If access is from within a VPC, is the endpoint policy allowing it?
- 6. KMS Encryption: If bucket uses SSE-KMS, does the user have kms:Decrypt permission on the key?
- 7. SCPs: In AWS Organisations, is a Service Control Policy blocking s3 actions?

■ *Interview tip: Use IAM Policy Simulator to test exact permissions without guessing. Use CloudTrail to see the exact denied API call and reason.*

Q6. You are facing latency issues in an EKS cluster. How do you troubleshoot?

Layer-by-layer approach:

- 1. Application layer: Check Datadog/APM traces — which service/endpoint is slow? p99 latency, error rate.
- 2. Pod resources: kubectl top pods — are pods CPU throttled or memory pressured? Check resource limits vs requests.
- 3. Node resources: kubectl top nodes — any node near 100% CPU/memory? Cluster Autoscaler not scaling fast enough?
- 4. Network: DNS resolution latency inside cluster? CoreDNS pod health. Check ndots:5 config causing excessive DNS lookups.
- 5. HPA behavior: Is HPA scaling fast enough? Check scaleUp stabilizationWindowSeconds.
- 6. Kube API latency: kubectl get --raw /metrics | grep apiserver_request_duration — high API server latency causes cascading delays.
- 7. etcd: High etcd latency → slow API server → slow scheduler. Check etcd disk I/O.

■ *Interview tip: Always start at the symptom (user-facing metric) and work inward. Don't jump to infra before checking app-layer traces.*

Q7. AWS Lambda is failing internally. What are your debugging steps?

- 1. CloudWatch Logs: Check /aws/lambda/ log group for stack traces, error messages, timeouts.
- 2. Check timeout: Is execution duration approaching the configured timeout limit (max 15min)?
- 3. Memory: Is memory usage near the configured limit? Increase if needed.
- 4. Cold starts: Provisioned Concurrency if latency is critical. Snap Start for Java runtimes.
- 5. VPC configuration: If Lambda is in a VPC, are the subnets private with NAT Gateway for outbound? Is security group allowing egress?
- 6. Concurrency limits: Account-level or function-level concurrency throttle? Check Lambda throttles metric in CloudWatch.
- 7. Dead Letter Queue (DLQ): For async invocations, failed events go to SQS/SNS DLQ. Check for accumulated messages.
- 8. X-Ray tracing: Enable active tracing to get distributed traces through Lambda and downstream services.

■ *Interview tip: For event-driven Lambda, check the trigger source (SQS, SNS, S3). Often the failure is upstream, not in the function itself.*

Q8. CI/CD pipeline is not able to connect to GitLab. What will you check?

- 1. Network: Can the Jenkins server reach GitLab's IP/domain? Test with curl or telnet on port 443/22.
- 2. Firewall/Security Group: Is outbound traffic on port 443 (HTTPS) or 22 (SSH) allowed from Jenkins?
- 3. Authentication: Is the GitLab access token expired? Check token scopes (needs read_repository / api).
- 4. SSH key: If using SSH, is the public key added to GitLab? Does the Jenkins user's known_hosts include GitLab?
- 5. GitLab webhook: Is the webhook secret correct? Is GitLab able to reach Jenkins' public IP for webhook push?
- 6. Proxy: If Jenkins is behind a corporate proxy, is HTTP_PROXY configured? Does it bypass internal GitLab?
- 7. GitLab runner: If using GitLab CI (not Jenkins), is the runner registered? Check runner status in GitLab UI.

■ *Interview tip: Check Jenkins system logs first (Manage Jenkins → System Log). The exact error (connection refused vs auth failure) tells you which layer.*

Q9. Kubernetes is not able to expose a pod. What could be the issue?

- 1. Label mismatch: Service selector labels don't match pod labels. kubectl describe svc → check Endpoints field (should show pod IPs).
- 2. Port mismatch: Service targetPort doesn't match containerPort in the pod spec.
- 3. Pod not running: kubectl get pods — is the pod in Running state? CrashLoopBackOff or Pending means it's not ready.
- 4. Readiness probe failing: Pod is running but not Ready. Service won't route traffic to pods failing readiness checks.
- 5. Network Policy: A NetworkPolicy is blocking ingress to the pod.
- 6. Ingress misconfiguration: Wrong service name/port in Ingress spec, or ingress controller not deployed.
- 7. NodePort range: NodePort must be 30000–32767. Using a port outside this range fails silently.

■ *Interview tip: kubectl get endpoints is the fastest check — empty endpoints = label mismatch or no ready pods.*

Q10. Pods are running but users are getting 503 errors. How do you debug?

503 = Service Unavailable. Traffic is reaching the load balancer/ingress but not getting a healthy backend response.

- 1. Endpoint check: kubectl get endpoints — are pod IPs listed? Empty = service can't find healthy pods.
- 2. Readiness probes: kubectl describe pod — are readiness probes passing? Failing probes remove pods from Service endpoints.
- 3. Ingress controller logs: kubectl logs -n ingress-nginx — look for upstream errors.
- 4. ALB target health: AWS Console → Target Group → check target health status. 'Unhealthy' = health check failing.
- 5. App-level: kubectl logs — is the app crashing on startup? Port mismatch?
- 6. Resource limits: Is the pod being OOMKilled right after starting (kubectl describe pod — check LastState)?

■ *Interview tip: 503 from ALB specifically means all targets are unhealthy or target group is empty. 503 from Nginx Ingress means it can't upstream to pods. Different sources, different fix.*

Q11. Terraform apply failed midway — how do you recover safely?

A mid-apply failure leaves Terraform state partially updated. Resources may exist in AWS that aren't fully tracked yet.

Recovery steps:

- 1. DO NOT run terraform destroy. You risk destroying partially created resources.
- 2. Check state: terraform show or terraform state list — see what got created.
- 3. Check AWS console: Verify actual resource state vs what Terraform thinks.
- 4. Fix the root cause: Address the error (IAM permission, quota limit, wrong config).
- 5. Re-run terraform apply: Terraform is idempotent — it will pick up from where it failed and only create missing resources.
- 6. If resource exists in AWS but not in state: terraform import to bring it under Terraform management.
- 7. State lock: If apply crashed, DynamoDB lock may still be held. Release with terraform force-unlock .

■ *Interview tip: Always use remote state with locking. A forced-unlock on a legitimately running apply causes corruption — confirm no apply is running first.*

Q12. How do you handle secrets securely across multiple environments?

Current bad practice to avoid:

- Base64 Kubernetes Secrets — NOT encrypted at rest by default. Anyone with etcd access can decode them.

Proper approach:

- AWS Secrets Manager: Store secrets centrally. IAM-controlled access per environment. Automatic rotation. Reference in K8s via External Secrets Operator.
- External Secrets Operator (ESO): Kubernetes operator that syncs secrets from AWS Secrets Manager / Vault into K8s Secret objects. Define ExternalSecret CRD → ESO fetches and creates the K8s Secret.
- HashiCorp Vault: Self-hosted secrets engine. Dynamic secrets, fine-grained policies, audit logs. Kubernetes auth method for pod-level access.
- Environment isolation: Separate AWS accounts or namespaces per environment. Each environment's pods can only access their own secrets via IAM role (IRSA).
- Never commit secrets to Git: Use .gitignore, git-secrets pre-commit hook, or GitGuardian scanning.

■ *Interview tip: Lead with External Secrets Operator + AWS Secrets Manager for K8s workloads. It's the current industry standard and directly addresses the gap in a plaintext-Secrets setup.*
